

Force-Field Optimization via AWH, Frozen-Bias Replay, and Information Geometry

Alexandre Blanco González

April 16, 2026

Contents

1	Overview	2
2	Frozen-Bias Reference Ensemble	2
3	MBAR Reweighting to an Arbitrary Lambda State	4
4	Discrete MBAR Weights Used by the Code	5
5	Endpoint Free-Energy Gradients	6
6	Bounded Coordinates	8
6.1	Charge Equilibration and Optimization	8
7	From Euclidean to Natural Gradient Descent	9
8	Why Fisher is the Metric Tensor	10
9	From the KL Expansion to a Covariance of Energy Gradients	11
10	Empirical Reweighting, ESS, and the Practical Fisher Estimate	13
11	Numerical Conditioning and Step Control	14
11.1	Diagonal Regularization	15
11.2	Jacobi Preconditioning	15
11.3	Eigentruncation	16
11.4	KL-Target Scaling and Confidence Moderation	16

12 Implemented Pipeline	17
12.1 Stage A: Cheap Readiness Screening	18
12.2 Stage B: Frozen-Bias Probe and Parity Validation	18
12.3 Frozen-Bias Production Artifacts	19
12.4 Offline Optimization and Step Acceptance	20
12.5 Current Scope and Default Example	21

1 Overview

This document describes the optimization pipeline implemented in the current repository state. The method alternates between adaptive AWH sampling, frozen-bias readiness probes, frozen-bias production collection, and offline reweighting-based optimization of bounded force-field parameters. The key point is that the optimization gradients are *not* obtained by differentiating the online AWH update rule. Instead, the adaptive AWH phase is used to construct a reliable reference ensemble. Once that reference ensemble has passed readiness checks, the bias is frozen and all thermodynamic estimators are evaluated offline from replayed frames. The idea behind using an extended ensemble as reference is to make the most out of the conformational richness that enhanced sampling simulations provide.

The repository currently supports a general coefficient-weighted thermodynamic cycle. The default example is ethanol hydration, with a solvent leg and a vacuum leg, but the mathematical structure is a sum over arbitrary legs with per-leg lambda schedules, ensembles, and coefficients.

2 Frozen-Bias Reference Ensemble

The implemented method separates two tasks:

1. adaptive AWH sampling, which is used to construct a reliable reference extended ensemble, and
2. offline thermodynamic estimation, which is performed only after that reference ensemble has been frozen.

The optimization gradients in the current code are not obtained by differentiating the online AWH update rule. Instead, the adaptive AWH phase is used only to produce a stationary reference distribution from which replay-based estimators can be evaluated cleanly.

Throughout this section, $i \in \Lambda_\ell$ denotes a generic lambda-state index, $m \in \Lambda_\ell$ denotes a target lambda state to which the stored frames are reweighted, and n indexes stored frames from the frozen reference trajectory. Target-state quantities are indexed by m in subscripts, while m appears as a function argument only when indicating evaluation at that same lambda state. We will repeatedly use two closely related reference objects: the full frozen extended ensemble over $(\mathbf{x}, V, i)$ and its conformational marginal over $(\mathbf{x}, V)$. The former is the stationary distribution produced by the frozen-bias AWH run, whereas the latter is the distribution that appears when frames are replayed into a chosen target lambda state.

Consider one thermodynamic leg ℓ with lambda states $i \in \Lambda_\ell$, inverse temperature β_ℓ , and, for NPT legs, pressure P_ℓ . The reduced potential of state i at parameters θ is defined as:

$$u_\ell(\mathbf{x}, V, i; \theta) = \beta_\ell U_\ell(\mathbf{x}, i; \theta) + \beta_\ell P_\ell V, \quad (1)$$

with the pressure-volume term omitted for legs that are not run at constant pressure.

Assume now we have run AWH until reasonable convergence has been achieved. We can then run a frozen-bias AWH simulation, that is, a simulation in which the bias and target lambda weights are no longer being updated, but the system can still jump between lambda-states. The reference Gibbs log-weights used are:

$$g_{\text{ref},\ell}(i) = f_\ell(i) + \log \rho_\ell(i), \quad (2)$$

where $f_\ell(i)$ is the frozen AWH free-energy estimate and $\rho_\ell(i)$ is the fixed target lambda distribution. The corresponding frozen extended-ensemble reference density therefore is:

$$p_{\text{ref},\ell}(\mathbf{x}, V, i) = \frac{\exp [g_{\text{ref},\ell}(i) - u_{\text{ref},\ell}(\mathbf{x}, V, i)]}{\mathcal{Z}_{\text{ref},\ell}}, \quad (3)$$

where $u_{\text{ref},\ell}$ denotes the reduced potential at the frozen reference parameterization and $\mathcal{Z}_{\text{ref},\ell}$ is the corresponding normalizing constant.

We can obtain the marginal of the conformational-space density by summing over all $i \in \Lambda_\ell$:

$$\begin{aligned} p_{\text{ref},\ell}(\mathbf{x}, V) &= \sum_{i \in \Lambda_\ell} p_{\text{ref},\ell}(\mathbf{x}, V, i) \\ &= \frac{1}{\mathcal{Z}_{\text{ref},\ell}} \sum_{i \in \Lambda_\ell} \exp [g_{\text{ref},\ell}(i) - u_{\text{ref},\ell}(\mathbf{x}, V, i)] \\ &= \frac{\mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V)}{\mathcal{Z}_{\text{ref},\ell}} \end{aligned} \quad (4)$$

Here

$$\mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V) = \sum_{i \in \Lambda_\ell} \exp [g_{\text{ref},\ell}(i) - u_{\text{ref},\ell}(\mathbf{x}, V, i)]$$

is the pointwise mixture normalizer obtained by summing over lambda states at fixed $(\mathbf{x}, V)$. It is not a global partition function; rather, it is the local normalizing factor that appears when the frozen extended ensemble is marginalized over the lambda coordinate.

3 MBAR Reweighting to an Arbitrary Lambda State

Having run an AWH simulation along a λ coordinate allows us to tap into a very rich conformational ensemble. A well-converged AWH bias guarantees (asymptotically) that the samples taken along λ follow a random walk, given a uniform target ρ , ensuring statistical independent samples. We can imagine a case where the free energy of interest can be computed just from the free-energies of the terminal λ -states, as is the case for the solvation free-energies. Instead of discarding frames that were not generated by the states of interest, we can still use them by reweighting them, to make the most of all the information we have.

This section concerns endpoint replay: the stored conformational frames from the frozen reference trajectory are re-evaluated as if they belonged to a chosen target lambda state m . The resulting weights therefore depend on that target-state index m and are the ones used later for endpoint free energies and endpoint free-energy gradients.

Let $m \in \Lambda_\ell$ be a target lambda state at which we want to evaluate a free energy or a free-energy gradient under the current parameter vector θ . We define the target-state partition function:

$$Q_{\ell,m}(\theta) = \int dV \int d\mathbf{x} \exp [-u_\ell(\mathbf{x}, V, m; \theta)]. \quad (5)$$

By definition, the corresponding reduced free energy is:

$$F_{\ell,m}(\theta) = -\log Q_{\ell,m}(\theta). \quad (6)$$

To connect $Q_{\ell,m}$ to the frozen reference ensemble, we multiply and divide the integrand in Equation (5) by the reference-mixture density of Equation (4) and obtain:

$$\begin{aligned}
Q_{\ell,m}(\theta) &= \int dV \int d\mathbf{x} \mathcal{Z}_{\text{ref},\ell} p_{\text{ref},\ell}(\mathbf{x}, V) \exp[-u_{\ell}(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V)] \\
&= \mathcal{Z}_{\text{ref},\ell} \mathbb{E}_{\text{ref},\ell} [\exp(-u_{\ell}(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V))] .
\end{aligned} \tag{7}$$

The expectation in Equation (7) is with respect to the reference conformational marginal $p_{\text{ref},\ell}(\mathbf{x}, V)$ of Equation (4), not with respect to the full extended distribution and not with respect to a candidate ensemble at updated parameters.

Taking minus the logarithm of Equation (7) gives:

$$F_{\ell,m}(\theta) = -\log \mathcal{Z}_{\text{ref},\ell} - \log \mathbb{E}_{\text{ref},\ell} [\exp(-u_{\ell}(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V))] . \tag{8}$$

The first term depends only on the frozen reference ensemble and is therefore a constant shared by every target state m of the same leg. We do not require that constant explicitly since only free-energy differences are used. It is therefore convenient to define the additive-constant-free quantity

$$\tilde{F}_{\ell,m}(\theta) = -\log \mathbb{E}_{\text{ref},\ell} [\exp(-u_{\ell}(\mathbf{x}, V, m; \theta) - \log \mathcal{Z}_{\text{mix},\ell}(\mathbf{x}, V))] , \tag{9}$$

so that

$$F_{\ell,m}(\theta) = -\log \mathcal{Z}_{\text{ref},\ell} + \tilde{F}_{\ell,m}(\theta) . \tag{10}$$

All endpoint differences within a leg depend only on $\tilde{F}_{\ell,m}(\theta)$ because the reference constant cancels.

4 Discrete MBAR Weights Used by the Code

This section turns the continuous endpoint-replay expectation above into the finite-sample weights that are actually formed by the implementation when evaluating endpoint observables.

Suppose the frozen reference trajectory for leg ℓ contains M_{ℓ} stored frames. Replacing the expectation in Equation (9) by the empirical average over those frames gives:

$$\tilde{F}_{\ell,m}(\theta) \approx -\log \left[\frac{1}{M_{\ell}} \sum_{n=1}^{M_{\ell}} w_{\ell,n,m}^{\text{rep}}(\theta) \right] , \tag{11}$$

where the unnormalized target-state replay weights are

$$w_{\ell,n,m}^{\text{rep}}(\theta) = \exp[-u_{\ell}(\mathbf{x}_n, V_n, m; \theta) - \log \mathcal{Z}_{\text{mix},\ell}(\mathbf{x}_n, V_n)] . \tag{12}$$

Because the prefactor $1/M_\ell$ is the same for every state of the same leg, the implementation evaluates

$$\tilde{F}_{\ell,m}(\theta) = -\log \sum_{n=1}^{M_\ell} w_{\ell,n,m}^{\text{rep}}(\theta), \quad (13)$$

which differs from Equation (11) only by the additive constant $\log M_\ell$ and therefore produces the same endpoint free-energy differences.

The normalized target-state replay weights used to reweight observables to the target state m are

$$\bar{w}_{\ell,n,m}^{\text{rep}}(\theta) = \frac{w_{\ell,n,m}^{\text{rep}}(\theta)}{\sum_{j=1}^{M_\ell} w_{\ell,j,m}^{\text{rep}}(\theta)}. \quad (14)$$

These are the weights implicitly formed by the current endpoint-gradient routine in the code. They replay each stored configuration frame $(\mathbf{x}_n, V_n)$ at the chosen target lambda state m . Equivalently, $\bar{w}_{\ell,n,m}^{\text{rep}}(\theta)$ is the normalized contribution of frame n when estimating an observable in target state m . In practice, this replay-weight family is the one used whenever the code evaluates endpoint free energies or endpoint free-energy gradients at a chosen target state.

The same machinery can be applied to every lambda state in the schedule to reconstruct a full replayed free-energy profile. Stage B readiness uses this full-profile reconstruction to compare MBAR-replayed profiles against the frozen AWH profile. Optimization, however, only needs the endpoint states. Later, when the natural-gradient machinery is introduced, a second weight family will appear. Those optimizer-side weights do not carry a target-state index m because they reweight each stored extended-state sample at its physically visited lambda state rather than replaying it into a chosen endpoint state.

5 Endpoint Free-Energy Gradients

The endpoint gradient estimator follows directly from the MBAR expression above. This section remains entirely in the physical parameters θ and uses the endpoint replay weights $\bar{w}_{\ell,n,m}^{\text{rep}}(\theta)$ associated with a chosen target lambda state m . Differentiating Equation (13) with respect to θ gives:

$$\begin{aligned}
\nabla_{\theta} \tilde{F}_{\ell,m}(\theta) &= -\frac{1}{\sum_{n=1}^{M_{\ell}} w_{\ell,n,m}^{\text{rep}}(\theta)} \sum_{n=1}^{M_{\ell}} \nabla_{\theta} w_{\ell,n,m}^{\text{rep}}(\theta) \\
&= -\frac{1}{\sum_{n=1}^{M_{\ell}} w_{\ell,n,m}^{\text{rep}}(\theta)} \sum_{n=1}^{M_{\ell}} [-w_{\ell,n,m}^{\text{rep}}(\theta) \nabla_{\theta} u_{\ell}(\mathbf{x}_n, V_n, m; \theta)] \\
&= \sum_{n=1}^{M_{\ell}} \bar{w}_{\ell,n,m}^{\text{rep}}(\theta) \nabla_{\theta} u_{\ell}(\mathbf{x}_n, V_n, m; \theta).
\end{aligned} \tag{15}$$

Since the pressure-volume term in Equation (1) does not depend on the force-field parameters,

$$\nabla_{\theta} u_{\ell}(\mathbf{x}_n, V_n, m; \theta) = \beta_{\ell} \nabla_{\theta} U_{\ell}(\mathbf{x}_n, m; \theta), \tag{16}$$

and therefore

$$\nabla_{\theta} \tilde{F}_{\ell,m}(\theta) = \sum_{n=1}^{M_{\ell}} \bar{w}_{\ell,n,m}^{\text{rep}}(\theta) \beta_{\ell} \nabla_{\theta} U_{\ell}(\mathbf{x}_n, m; \theta). \tag{17}$$

This is the precise sense in which the MBAR weights are used to compute endpoint gradients in the implementation: every stored frame is replayed at the lambda state of interest, and the corresponding force-field gradient at that endpoint is averaged with the normalized target-state replay weights $\bar{w}_{\ell,n,m}^{\text{rep}}(\theta)$. Throughout this section, the target-state index m always denotes the lambda state at which the stored frame is being evaluated, not necessarily the lambda state that was physically visited when that frame was generated.

The endpoint free energy of a leg is the coupled-minus-decoupled difference

$$\Delta G_{\ell}(\theta) = \tilde{F}_{\ell,\text{coupled}}(\theta) - \tilde{F}_{\ell,\text{decoupled}}(\theta), \tag{18}$$

and its gradient is

$$\nabla_{\theta} \Delta G_{\ell}(\theta) = \nabla_{\theta} \tilde{F}_{\ell,\text{coupled}}(\theta) - \nabla_{\theta} \tilde{F}_{\ell,\text{decoupled}}(\theta). \tag{19}$$

For a coefficient-weighted thermodynamic cycle,

$$\Delta G_{\text{cyc}}(\theta) = \Delta G_{\text{std}} + \sum_{\ell} c_{\ell} \Delta G_{\ell}(\theta), \tag{20}$$

and the residual against experiment is

$$\mathcal{E}(\theta) = \Delta G_{\text{cyc}}(\theta) - \widetilde{\Delta G}_{\text{exp}}. \tag{21}$$

The code applies a Huber derivative to this residual, so the optimization gradient is actually:

$$\nabla_{\theta} \mathcal{L} = \frac{d\mathcal{L}}{d\mathcal{E}} \nabla_{\theta} \Delta G_{\text{cyc}}(\theta). \quad (22)$$

6 Bounded Coordinates

However, the optimizer does not update unconstrained physical parameters directly. Each trainable parameter is represented by a latent coordinate ϕ_i and mapped into a bounded interval $[\theta_{i,\min}, \theta_{i,\max}]$ through a shifted sigmoid transform,

$$\theta_i(\phi_i) = \theta_{i,\min} + (\theta_{i,\max} - \theta_{i,\min}) \sigma(k\phi_i), \quad (23)$$

where k is a configurable slope and σ is the logistic function. The chain rule then gives:

$$\nabla_{\phi_i} U = \frac{d\theta_i}{d\phi_i} \nabla_{\theta_i} U, \quad (24)$$

so all optimization-space gradients and Fisher matrices are assembled in latent coordinates. From this point onward, whenever a reduced potential, free energy, or observable gradient is written as a function of ϕ , this denotes composition with the bounded map in Equation (23); for example, $u_{\ell}(\mathbf{x}, V, i; \phi)$ is shorthand for $u_{\ell}(\mathbf{x}, V, i; \theta(\phi))$. Accordingly, the endpoint replay formulas above are written in the physical parameters θ , whereas the optimizer-side scores, Fisher matrices, and candidate-ensemble reweighting factors below are written in the latent coordinates ϕ .

6.1 Charge Equilibration and Optimization

In addition to Lennard-Jones parameters, the current implementation supports the joint optimization of partial charges through a constrained charge-equilibration model. Instead of optimizing the partial charges q_i directly, the method treats the electronegativity χ_i and the hardness η_i of each atom type (or class) as the trainable physical parameters.

For a given molecule with a target net charge Q_{mol} , the partial charge q_i of each trainable atom i is obtained by equalizing the effective electronegativity across all atoms in the molecule, subject to the total charge constraint. The charge is given by:

$$q_i = -\frac{\chi_i + \lambda_{\text{mol}}}{\eta_i}, \quad (25)$$

where λ_{mol} is the molecule-specific Lagrange multiplier ensuring that the sum of the charges equals the target net charge. To satisfy

$$\sum_{i \in \text{trainable}} q_i = Q_{\text{mol}} - \sum_{j \in \text{fixed}} q_j, \quad (26)$$

the Lagrange multiplier is analytically solved as:

$$\lambda_{\text{mol}} = - \frac{Q_{\text{mol}} - \sum_{j \in \text{fixed}} q_j + \sum_{i \in \text{trainable}} \frac{\chi_i}{\eta_i}}{\sum_{i \in \text{trainable}} \frac{1}{\eta_i}}. \quad (27)$$

By parameterizing the force field in terms of χ and η , the constraint Q_{mol} is strictly enforced at every step of the optimization without requiring constrained gradient-descent manifolds. The physical bounds are maintained by applying a strictly positive transform to the latent coordinates for hardness, ensuring $\eta_i \geq \eta_{\text{min}} > 0$ to avoid singularities, while electronegativity coordinates remain lightly constrained or unbounded.

7 From Euclidean to Natural Gradient Descent

At this point the reason for introducing a metric tensor in the optimization step can be stated explicitly. The endpoint replay formulas above determine the loss gradient, and after applying the chain rule of Equation (24) that gradient lives in the latent coordinates ϕ . The remaining question is then how to measure the size of a step in a way that reflects how strongly the induced ensemble changes. If one ignored the geometry of the probability distributions and performed ordinary Euclidean gradient descent in latent parameter space, the step would be:

$$\Delta\phi_{\text{Euc}} = -\eta\nabla_{\phi}\mathcal{L}. \quad (28)$$

This treats all coordinate directions in ϕ -space as geometrically equivalent. That assumption grossly fails for force-field optimization. Different parameters perturb the sampled distribution with very different strength: a small change in one parameter can produce a large statistical distortion (*e.g.*, a Lennard-Jones σ), while a numerically equivalent change in another parameter can have only a modest effect (*e.g.*, a torsion force constant). In other words, the relevant geometry is not the Euclidean geometry of the coordinates themselves, but the geometry of the manifold of probability distributions induced by those coordinates.

The natural gradient is the steepest-descent direction on that statistical manifold. If $\mathcal{I}(\phi)$ is used as the local metric tensor, then the corresponding natural-gradient update is:

$$\Delta\phi_{\text{nat}} = -\eta\mathcal{I}(\phi)^{-1}\nabla_{\phi}\mathcal{L}. \quad (29)$$

In our setting, the appropriate local metric is the Fisher information matrix. We use this matrix as a local description of the manifold curvature, so that the update direction is the local steepest-descent direction consistent with distribution space rather than raw coordinate space. The code first builds a latent-space loss gradient, then preconditions it by the empirical Fisher matrix to obtain a natural-gradient-like base step. That step is then stabilized by diagonal normalization, eigentruncation, KL-target scaling, and a line search. The KL derivation below explains why this is the correct local geometry to use.

8 Why Fisher is the Metric Tensor

The endpoint replay formulas above determine which direction lowers the objective, but the optimizer also needs a notion of step size in distribution space. For that purpose we compare the current candidate ensemble at ϕ with a nearby candidate ensemble at $\phi + \delta\phi$. The relevant comparison is carried out on the extended state $(\mathbf{x}, V, i)$, because the optimizer-side Fisher estimate is built from stored extended-state samples at their visited lambda states.

The Fisher information matrix appears as the first non-zero local approximation to the Kullback-Leibler divergence between nearby candidate ensembles. This is precisely why the current optimizer uses it as a metric tensor: it measures how large a parameter step is in the space of probability distributions, not in the raw coordinate system of the parameters.

For a fixed frozen reference Gibbs scheme $g_{\text{ref},\ell}$ and current latent parameter vector ϕ , we define the candidate extended-state density:

$$p_{\phi}(\mathbf{x}, V, i) = \frac{\exp[g_{\text{ref},\ell}(i) - u_{\ell}(\mathbf{x}, V, i; \phi)]}{\mathcal{Z}_{\ell}(\phi)}. \quad (30)$$

Here the dependence on ϕ enters only through the reduced potential, understood through the map $\theta = \theta(\phi)$ introduced above; the frozen AWH Gibbs log-weights are held fixed. Thus p_{ϕ} is the candidate ensemble induced by the current parameters under the same frozen lambda-biasing scheme. For readability, the leg index is suppressed on p_{ϕ} itself in the expressions below; each of them is understood legwise before leg contributions are combined. We can now compare p_{ϕ} with a nearby candidate $p_{\phi+\delta\phi}$. The KL divergence from the current ensemble to the proposed ensemble is defined as:

$$D_{\text{KL}}(p_{\phi} \parallel p_{\phi+\delta\phi}) = \mathbb{E}_{\phi}[\log p_{\phi} - \log p_{\phi+\delta\phi}]. \quad (31)$$

If we now Taylor-expand the log density of the proposed distribution around ϕ , we obtain:

$$\log p_{\phi+\delta\phi} = \log p_{\phi} + \delta\phi^{\top} \mathbf{s}_{\phi} + \frac{1}{2} \delta\phi^{\top} H_{\phi} \delta\phi + O(\|\delta\phi\|^3), \quad (32)$$

where

$$\mathbf{s}_{\phi} = \nabla_{\phi} \log p_{\phi} \quad (33)$$

is the score vector and

$$H_{\phi} = \nabla_{\phi}^2 \log p_{\phi} \quad (34)$$

is the Hessian of the log density.

Substituting Equation (32) into Equation (31) yields:

$$D_{\text{KL}}(p_{\phi} \parallel p_{\phi+\delta\phi}) = -\delta\phi^{\top} \mathbb{E}_{\phi}[\mathbf{s}_{\phi}] - \frac{1}{2} \delta\phi^{\top} \mathbb{E}_{\phi}[H_{\phi}] \delta\phi + O(\|\delta\phi\|^3). \quad (35)$$

Under commutability of differentiation and integration, the score expectation vanishes:

$$\mathbb{E}_{\phi}[\mathbf{s}_{\phi}] = \int p_{\phi} \nabla_{\phi} \log p_{\phi} = \int \nabla_{\phi} p_{\phi} = \nabla_{\phi} \int p_{\phi} = \nabla_{\phi} 1 = 0. \quad (36)$$

Therefore, the linear term disappears and the *first nonzero* contribution to the local KL divergence is second order:

$$D_{\text{KL}}(p_{\phi} \parallel p_{\phi+\delta\phi}) = \frac{1}{2} \delta\phi^{\top} \mathcal{I}(\phi) \delta\phi + O(\|\delta\phi\|^3), \quad (37)$$

where

$$\mathcal{I}(\phi) = -\mathbb{E}_{\phi}[H_{\phi}] \quad (38)$$

is the Fisher information matrix, and Equation (37) is the reason the Fisher matrix is the natural metric tensor: its quadratic form is the local statistical distance induced by KL divergence.

9 From the KL Expansion to a Covariance of Energy Gradients

This section turns the abstract KL-based definition of the Fisher information matrix into the covariance form that the implementation can estimate from stored samples. Under the standard Fisher regularity conditions, namely; that the support of the probability density does not depend on ϕ , the parameter space is open,

the log-density is twice continuously differentiable with respect to ϕ , the integration and differentiation operators commute, and the Fisher Information Matrix is positive-definite, we have:

$$\mathcal{I}(\phi) = \mathbb{E}_\phi [\mathbf{s}_\phi \mathbf{s}_\phi^\top]. \quad (39)$$

To turn this abstract definition into the object actually used by the code, we explicitly derive the expression for the score of the frozen Gibbs bias in Equation (30). Taking the logarithm of that same Equation gives:

$$\log p_\phi(\mathbf{x}, V, i) = g_{\text{ref}, \ell}(i) - u_\ell(\mathbf{x}, V, i; \phi) - \log \mathcal{Z}_\ell(\phi). \quad (40)$$

then, differentiating it with respect to ϕ yields:

$$\begin{aligned} \nabla_\phi \log p_\phi(\mathbf{x}, V, i) &= \mathbf{s}_\phi(\mathbf{x}, V, i) \\ &= -\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) - \nabla_\phi \log \mathcal{Z}_\ell(\phi). \end{aligned} \quad (41)$$

Now,

$$\begin{aligned} \nabla_\phi \log \mathcal{Z}_\ell(\phi) &= \frac{1}{\mathcal{Z}_\ell(\phi)} \nabla_\phi \mathcal{Z}_\ell(\phi) \\ &= \frac{1}{\mathcal{Z}_\ell(\phi)} \sum_i \int dV \int d\mathbf{x} [-\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi)] \\ &\quad \times \exp [g_{\text{ref}, \ell}(i) - u_\ell(\mathbf{x}, V, i; \phi)] \\ &= -\mathbb{E}_\phi [\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi)]. \end{aligned} \quad (42)$$

and substituting Equation (42) into Equation (41) gives:

This score is a function of the full extended state $(\mathbf{x}, V, i)$, because the candidate density p_ϕ itself is an extended-state distribution.

$$\mathbf{s}_\phi(\mathbf{x}, V, i) = -\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi) + \mathbb{E}_\phi [\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi)]. \quad (43)$$

This is already a centered quantity, so the Fisher matrix is the covariance of reduced-potential gradients:

$$\mathcal{I}(\phi) = \text{Cov}_\phi (\nabla_\phi u_\ell(\mathbf{x}, V, i; \phi)). \quad (44)$$

At this stage the expectation is still under the candidate ensemble p_ϕ , so the quantity cannot yet be evaluated directly from frozen-reference samples without a further reweighting step, introduced in the next section. Finally, because $u_\ell(\mathbf{x}, V, i; \phi) = \beta_\ell U_\ell(\mathbf{x}, i; \phi) + \beta_\ell P_\ell V$ and the $P_\ell V$ term does not depend on ϕ ,

$$\nabla_{\phi} u_{\ell}(\mathbf{x}, V, i; \phi) = \beta_{\ell} \nabla_{\phi} U_{\ell}(\mathbf{x}, i; \phi), \quad (45)$$

which leads to the form used conceptually by the implementation:

$$\mathcal{I}(\phi) = \text{Cov}_{\phi}(\beta_{\ell} \nabla_{\phi} U_{\ell}(\mathbf{x}, i; \phi)). \quad (46)$$

Thus, the Fisher matrix is not an arbitrary preconditioner. It is the covariance of the reduced-potential score vectors and the local metric tensor induced by KL divergence.

10 Empirical Reweighting, ESS, and the Practical Fisher Estimate

The formulas above are expectations under the current candidate distribution p_{ϕ} , but the stored frames come from the frozen reference distribution $p_{\text{ref},\ell}$. The code therefore reweights the reference frames into the current candidate ensemble. These weights are distinct from the target-state replay weights introduced earlier. Earlier, each stored configuration frame $(\mathbf{x}_n, V_n)$ was replayed at an arbitrarily chosen target lambda state m , producing weights that depend on m and are used for endpoint free energies and endpoint free-energy gradients. Here, instead, each stored extended-state sample $(\mathbf{x}_n, V_n, i_n)$ is reweighted into the current candidate extended ensemble at its physically visited lambda state i_n , so the weights carry no target-state index m and are functions of the latent parameters ϕ .

For the full extended state $(\mathbf{x}, V, i)$,

$$\frac{p_{\phi}(\mathbf{x}, V, i)}{p_{\text{ref},\ell}(\mathbf{x}, V, i)} = \frac{\mathcal{Z}_{\text{ref},\ell}}{\mathcal{Z}_{\ell}(\phi)} \exp[-u_{\ell}(\mathbf{x}, V, i; \phi) + u_{\text{ref},\ell}(\mathbf{x}, V, i)] \quad (47)$$

Because the frozen Gibbs bias $g_{\text{ref},\ell}(i_n)$ appears in both numerator and denominator, it cancels exactly. For a stored frame n with visited lambda state i_n , the unnormalized candidate-ensemble reweighting factor used by the optimizer is therefore:

$$w_{\ell,n}^{\text{ens}}(\phi) \propto \exp[-u_{\ell}(\mathbf{x}_n, V_n, i_n; \phi) + u_{\text{ref},\ell}(\mathbf{x}_n, V_n, i_n)]. \quad (48)$$

For NPT legs, the $P_{\ell}V$ term also cancels because the same pressure is used in the reference and candidate ensembles. The normalized candidate-ensemble weights therefore are:

$$\overline{w}_{\ell,n}^{\text{ens}}(\phi) = \frac{w_{\ell,n}^{\text{ens}}(\phi)}{\sum_{j=1}^{M_{\ell}} w_{\ell,j}^{\text{ens}}(\phi)}, \quad (49)$$

These normalized candidate-ensemble weights approximate expectations under $p_\phi(\mathbf{x}, V, i)$ using samples drawn from $p_{\text{ref},\ell}(\mathbf{x}, V, i)$. They are used for optimizer-side quantities such as the effective sample size and the empirical Fisher estimate; they are not the replay weights used for endpoint free energies or endpoint gradients. The effective sample size can be estimated as:

$$\text{ESS}_\ell(\phi) = \frac{1}{\sum_{n=1}^{M_\ell} \bar{w}_{\ell,n}^{\text{ens}}(\phi)^2}. \quad (50)$$

Unlike the endpoint replay gradients above, which use $\nabla_\theta U_\ell(\mathbf{x}_n, m; \theta)$ evaluated at a chosen target state m , the quantity below is the samplewise realization of the score in latent coordinates and is evaluated at the physically visited state i_n . We calculate the empirical latent-space score vector for frame n at the physically visited lambda state as:

$$\mathbf{s}_{\ell,n}(\phi) = \beta_\ell \nabla_\phi U_\ell(\mathbf{x}_n, i_n; \phi), \quad (51)$$

and the weighted empirical Fisher matrix as:

$$\hat{\mathcal{I}}_\ell(\phi) = \sum_{n=1}^{M_\ell} \bar{w}_{\ell,n}^{\text{ens}}(\phi) (\mathbf{s}_{\ell,n}(\phi) - \bar{\mathbf{s}}_\ell(\phi)) (\mathbf{s}_{\ell,n}(\phi) - \bar{\mathbf{s}}_\ell(\phi))^\top, \quad (52)$$

where

$$\bar{\mathbf{s}}_\ell(\phi) = \sum_{n=1}^{M_\ell} \bar{w}_{\ell,n}^{\text{ens}}(\phi) \mathbf{s}_{\ell,n}(\phi). \quad (53)$$

The full thermodynamic cycle Fisher matrix is the sum of the leg contributions in the shared latent parameter space.

11 Numerical Conditioning and Step Control

The raw empirical Fisher matrix $\hat{\mathcal{I}}(\phi)$ is often ill-conditioned. This is a practical consequence of finite sampling: some parameter directions, or linear combinations thereof, may only weakly affect the sampled ensemble, so the corresponding eigenvalues are very small and the formal natural-gradient step becomes numerically unstable. The role of this section is therefore not to introduce a new geometric identity, but to describe how the implementation converts the empirical Fisher estimate into a robust update. The current code applies four postprocessing stages: diagonal regularization, Jacobi preconditioning, eigentruncation, and KL-target scaling.

11.1 Diagonal Regularization

A small regularization term is added to the diagonal of the empirical Fisher matrix. In the current implementation, this term is motivated by the quadratic reference-model penalty used to discourage the parameters from drifting too far from their starting values. In latent coordinates, that penalty contributes an additional diagonal curvature, so if λ_p is the penalty strength for parameter p , the matrix is updated as:

$$\hat{\mathcal{I}}_{pp} \leftarrow \hat{\mathcal{I}}_{pp} + \lambda_p \left(\frac{d\theta_p}{d\phi_p} \right)^2 \quad (54)$$

This term should be viewed as a numerical curvature regularizer motivated by the reference penalty, rather than as a separate Fisher identity. Its practical effect is to lift poorly informed diagonal directions and reduce the risk of unstable steps. If the added diagonal contribution is strictly positive in all retained directions, the regularized matrix becomes strictly positive definite; more generally, it still improves conditioning even when that stronger statement does not hold globally.

11.2 Jacobi Preconditioning

Before eigentruncation, the code applies a diagonal normalization, also known as Jacobi preconditioning. Its main purpose is to make the truncation criterion insensitive to the arbitrary physical scales and units of the force-field parameters (e.g., kJ/mol vs. nm vs. elementary charge). Let D be the diagonal matrix of the empirical Fisher matrix, $D = \text{diag}(\hat{\mathcal{I}})$. The preconditioned Fisher matrix and scaled gradient are formed as:

$$\hat{\mathcal{I}}_{\text{corr}} = D^{-1/2} \hat{\mathcal{I}} D^{-1/2}, \quad \mathbf{g}_{\text{scaled}} = D^{-1/2} \nabla_{\phi} \mathcal{L} \quad (55)$$

In the limit of exact inversion, this diagonal rescaling does not change the final natural-gradient step, because the factors of D cancel out when the matrix is inverted and mapped back to the original coordinates. Its practical importance appears once eigentruncation is introduced: after rescaling, the retained and discarded directions are determined by the correlation structure of the empirical Fisher matrix rather than by the raw magnitudes induced by parameter units.

Without Jacobi preconditioning, eigentruncation could discard physically relevant directions simply because those coordinates have small numerical curvature in their native units. After rescaling, each coordinate is measured relative to its own local diagonal curvature, making $\hat{\mathcal{I}}_{\text{corr}}$ behave like a correlation-like matrix. The subsequent truncation then preferentially removes directions that are weakly informed by the sampled reference ensemble, rather than directions that merely happen to be expressed in small numerical units.

11.3 Eigentruncation

The preconditioned matrix $\widehat{\mathcal{I}}_{\text{corr}}$ is then decomposed into its eigenvalues λ_j and eigenvectors $\mathbf{v}_j$. Directions corresponding to eigenvalues below a relative tolerance ϵ_{tol} (typically 10^{-5} of the maximum eigenvalue) are treated as weakly informed or nearly redundant and are truncated. The regularized inverse is constructed as:

$$\widehat{\mathcal{I}}_{\text{corr}}^\dagger = \sum_{\lambda_j > \epsilon_{\text{tol}} \lambda_{\text{max}}} \frac{1}{\lambda_j} \mathbf{v}_j \mathbf{v}_j^\top. \quad (56)$$

This eigentruncation prevents the optimizer from taking very large steps along directions for which the empirical Fisher provides little reliable curvature information.

11.4 KL-Target Scaling and Confidence Moderation

The base latent-space step is then computed as

$$\Delta\phi_{\text{base}} = D^{-1/2} \widehat{\mathcal{I}}_{\text{corr}}^\dagger \mathbf{g}_{\text{scaled}}.$$

The optimizer next imposes a trust-region bound in distribution space. Because the first nonzero term in the local KL expansion is $\frac{1}{2} \delta\phi^\top \mathcal{I}(\phi) \delta\phi$, the code uses the quadratic KL model to rescale the base step so that the predicted divergence does not exceed a target value δ_{KL} :

$$\Delta\phi = \alpha \Delta\phi_{\text{base}}, \quad \text{with } \alpha = \min \left(1, \sqrt{\frac{2\delta_{\text{KL}}}{\Delta\phi_{\text{base}}^\top \widehat{\mathcal{I}} \Delta\phi_{\text{base}}}} \right) \quad (57)$$

The target δ_{KL} is itself moderated by a confidence factor $s_{\text{conf}} \in [s_{\text{min}}, 1]$. In the implementation, this factor is obtained by comparing the optimization-relevant quantities estimated from the first and second halves of the production data, such as endpoint free energies and gradients. Large split-half disagreement is interpreted as evidence that the current replay estimates are noisy or insufficiently converged, in which case s_{conf} is reduced. This shrinks the trust region and makes the update more conservative. This confidence moderation is therefore an additional robustness heuristic layered on top of the KL trust region, not a separate geometric identity.

The empirical Fisher matrix is therefore the operator that turns the Euclidean loss gradient into a natural-gradient-like base step, after which the optimizer applies the stabilization steps described above. The full logic is:

1. replay the frozen reference frames to obtain endpoint free energies and endpoint gradients,

2. assemble the latent-space loss gradient from those endpoint contributions,
3. estimate the empirical Fisher matrix from reweighted score vectors at the visited lambda states,
4. use that Fisher matrix as the local metric tensor of the statistical manifold,
5. stabilize and bound the proposed update through diagonal regularization, scale-invariant truncation, and the quadratic KL model in Equation (57).

This is why a KL target, rather than a raw Euclidean step length, is the natural quantity to bound. A step that is small in Euclidean coordinates can still be large in distribution space if it distorts the ensemble strongly, while a step that is large in a weakly constrained coordinate may still be statistically safe. The Fisher metric captures that distinction directly. In that sense, the endpoint replay machinery determines which direction lowers the objective, whereas the empirical Fisher determines how far the optimizer can move in that direction before the induced ensemble changes too much.

12 Implemented Pipeline

The current code organizes the method as a macro-epoch loop. At the beginning of each macro epoch, the pipeline constructs one AWH simulation per thermodynamic leg with the current physical parameters. Restart coordinates and velocities may be reused from the previous macro epoch. The AWH bias is reused only when restart continuity is preserved; a cold restart from the input structure does not reuse a previous bias profile. For each leg, the macro epoch proceeds through four stages:

1. optional rewarm of a reused simulation state,
2. readiness-gated AWH sampling,
3. frozen-bias production collection,
4. offline optimization from the frozen production artifacts.

The optimizer never consumes data from an actively adapting AWH bias.

12.1 Stage A: Cheap Readiness Screening

Stage A is a low-cost check applied after each AWH block. It does not attempt to estimate free energies directly. Instead, it asks whether the adaptive AWH run is mixed and statistically mature enough to justify a frozen-bias probe. The current implementation monitors:

- recent mean bias-change magnitude,
- lambda-history effective sample size,
- Molly’s linear-stage effective sample count N_{eff} ,
- full round trips between the coupled and decoupled states,
- recent endpoint occupancy,
- solvent-tail occupancy floors for late Lennard-Jones states in the solvent leg.

Only after a leg passes these Stage A criteria for a required stable streak does the pipeline launch a Stage B probe. Repeated Stage B failures can increase the required stable streak or introduce cooldown periods before the next probe.

12.2 Stage B: Frozen-Bias Probe and Parity Validation

Stage B is the decisive readiness check. The current leg is cloned into a frozen reference simulation, the AWH bias is held fixed, bias-update accumulators are cleared, and a short probe trajectory is generated. The probe frames are then processed offline:

1. discard an initial fraction of the probe,
2. apply stride and minimum/maximum retained-frame rules,
3. rebuild CPU replay templates for all lambda states,
4. evaluate each retained frame under every state Hamiltonian.

The resulting energy matrix is used to compute two independent Stage B diagnostics.

Split-half convergence. The retained frames are split into temporal halves. A coupled-minus-decoupled free energy is computed independently from each half, and the split gap is the absolute difference between those two estimates.

MBAR/AWH parity. The same retained frames are used to reconstruct a full lambda free-energy profile from the frozen reference mixture. That replayed profile is aligned to the coupled endpoint and compared against the frozen AWH profile that generated the frames. The implementation tracks:

- the raw parity gap over all states,
- the supported parity gap over states whose reweighting ESS exceeds a configured support threshold,
- a separate endpoint parity gap,
- minimum support coverage over the full lambda schedule.

The default gate is therefore *support-aware* but not support-blind: a leg must satisfy split convergence, supported parity, endpoint parity, and support coverage simultaneously.

Split-only continuation. The current code distinguishes true split-only failures from genuine parity failures. If split convergence fails while supported parity, endpoint parity, and support coverage all still pass, the pipeline does *not* return immediately to adaptive AWH sampling. Instead, it continues the same frozen probe in place, appending one more segment under the unchanged frozen bias. The raw logger history is concatenated and the entire frame-selection procedure is rerun on the combined probe so that discard, stride, and frame-cap rules remain consistent. This continuation ends as soon as the failure mode is no longer split-only or the configured segment budget is exhausted.

Retry policy. When Stage B fails for lack of support or split convergence, the next probe is grown. When it fails on parity, the probe length is held fixed and the leg is sent back to Stage A so the adaptive bias can continue to improve. Near-pass behavior is leg-specific: by default, the solvent leg keeps the same probe length on a near-pass rather than shrinking it, while the vacuum leg may still shrink.

12.3 Frozen-Bias Production Artifacts

Once every leg in the cycle has passed Stage B, the pipeline launches a second frozen-bias simulation for production collection. For each leg, the current code stores:

- the logged coordinates, active lambda history, and, where relevant, volume history,

- the frozen bias metadata,
- precomputed replay neighbor data,
- an ensemble-evaluation cache for CPU replay,
- the reference energy matrix $u_{\text{ref},\ell}$ over all store frames and all lambda states.

These production artifacts are the only data used by the optimizer.

12.4 Offline Optimization and Step Acceptance

The optimizer evaluates the current candidate parameter vector on the frozen production artifacts and assembles three objects:

1. the coefficient-weighted cycle free energy,
2. the coefficient-weighted cycle gradient in latent space,
3. the empirical Fisher matrix in latent space.

The base step is formed by diagonal preconditioning, eigentruncation, KL-target scaling, and an infinity-norm cap on the latent update. The line search then proposes

$$\phi_{\text{prop}} = \phi_{\text{current}} - \alpha \Delta\phi, \quad (58)$$

starting from $\alpha = 1$ and halving repeatedly until two conditions hold:

- every leg remains above its effective sample size threshold,
- the residual improves enough to satisfy the acceptance test.

The residual test is not a simple monotonic decrease. The current implementation accepts a proposal only if the new residual lies inside a tolerance-adjusted window based on the current residual magnitude, and the required improvement is made stricter when the production split-half confidence is poor.

Split-half confidence scaling. The production artifacts are also split deterministically into first and second temporal halves. The code recomputes endpoint free energies and gradients on each half and forms three disagreement measures:

- the maximum per-leg endpoint ΔG disagreement,
- the disagreement in the full cycle contribution,
- the disagreement in the cycle gradient vector.

These are compressed into a confidence factor

$$s_{\text{conf}} = \max \left(s_{\text{min}}, \frac{1}{1 + \kappa \delta_{\text{max}}} \right), \quad (59)$$

where δ_{max} is the largest disagreement signal. The same split-half analysis also adds an extra residual-improvement requirement proportional to the cycle disagreement before a line-search step is accepted.

Inner-loop exits. If the accepted line-search step becomes too small, if the line search repeatedly hits a tiny α , or if the configured inner-epoch cap is reached, the code restores the best state seen inside that macro epoch and returns to fresh resimulation. A fresh macro-epoch Stage B failure does not trigger a hard rejection of the previously accepted parameter set; it simply prevents additional optimization until new readiness is established.

12.5 Current Scope and Default Example

The repository now supports a general thermodynamic cycle with arbitrary legs, coefficients, per-leg lambda schedules, and per-leg ensembles. The default example remains ethanol hydration. In that default setup:

- the solvent leg uses an NPT expanded ensemble with a staged, charge-first and Lennard-Jones-second lambda schedule concentrated in the low-Lennard-Jones tail,
- the vacuum leg uses a simpler non-periodic leg with no pressure-volume term,
- the optimization can jointly train Lennard-Jones parameters and partial charges. When charge training is enabled, the code uses a constrained charge-equilibration algorithm (predicting electronegativity and hardness per atom class) to solve for partial charges under exact molecular net-charge constraints before evaluating the force-field energy,

- optimization is solute-only unless solvent parameters are explicitly enabled.

This is therefore no longer a speculative milestone roadmap. The implemented method is a readiness-gated, frozen-bias replay pipeline with support-aware Stage B validation, bounded latent-coordinate optimization, KL-limited natural-gradient steps, and deterministic confidence moderation from split-half production artifacts.